

Robotics Rover

Chain K — Hardware & Systems Foundations · Full written lesson

Every other build in this chain reads the world or displays to it. A rover acts on it — and motors are a genuinely different problem: they draw far more current than logic circuits, need their own driver stage, and closing a feedback loop is a different kind of programming than reading a sensor value.

Lesson 1 — H-Bridges, PWM, and Choosing a Driver IC

A motor is a fundamentally different load than an LED or a sensor: it draws far more current, and a microcontroller pin can't drive it directly without frying itself.

- An **H-bridge** is four switches arranged so current can flow through a motor in either direction. A low-current logic signal from the microcontroller controls those switches, letting it spin a high-current motor forward, backward, or stop — and reverse it electronically, never by physically swapping wires.
- **PWM** (pulse-width modulation) rapidly switches the drive signal on and off; the duty cycle controls the effective average voltage the motor sees — real speed control, with no analog voltage regulator needed.
- **Driver IC choice matters**: a TB6612FNG (MOSFET-based) has a much lower voltage drop than an older L298N, so it wastes less power as heat and doesn't need a heatsink under normal load.
- Motors draw far more current **stalled** than running freely — size the driver and battery for stall current, not typical running current.

The trap: sizing a driver board or battery for a motor's typical running current instead of its much higher stall current. A motor that draws 200mA free-running can draw well over 1A stalled — exactly the moment a load gets stuck, which is precisely when you can't afford the driver to fail.

Worked example: an L298N's roughly 1.4V drop per channel gets wasted as heat at real motor currents, which is exactly why it needs a heatsink under load — a TB6612FNG's much lower drop is the concrete, measurable reason it runs cooler.

Lesson 2 — Why Motors Need Their Own Battery, Sharing Only Ground

This single wiring decision prevents the single most common rover failure — and understanding exactly why it works is more useful than just following the rule.

- A motor's start-up current is a real, sudden spike. If the motor and the microcontroller share the same battery or regulator, that spike sags the shared voltage rail — and if it drops below the microcontroller's minimum operating voltage, the microcontroller actually resets (a brownout).
- The fix is two separate power domains: the motor battery powers only the driver's motor-side terminals, a separate supply powers the logic side, and the two rails meet only at ground.

- **Flyback diodes** protect against a related problem: when a motor's current is suddenly interrupted, the motor's own inductance generates a voltage spike (back-EMF) that can exceed the driver's rated voltage. Most breakout boards include these; bare H-bridge ICs may not.

The trap: a driver board with no flyback diodes, letting motor back-EMF damage the driver over time — worth explicitly checking for on a bare IC.

Worked example: the exact "#1 rover failure" — motor start-up current brownout-ing the microcontroller mid-command — is prevented entirely by the two-power-domain wiring, with zero additional code or firmware changes needed.

Lesson 3 — Chassis Assembly and Sensing for Line-Following

The mechanical build and the sensors that feed the control loop both deserve real attention — a great control algorithm can't fix a chassis that binds or a sensor mounted at the wrong height.

- A chassis with metal gearboxes holds up far better over repeated use than an all-plastic one, which can strip teeth under load.
- An **IR line sensor** emits infrared light and measures how much bounces back. A light surface reflects far more IR back than a dark line does — that reflectance difference is the actual signal a line-follower reads.
- **Sensor mounting height matters**: typically 5-10mm above the surface. Too high and the reflected signal is weak and noisy; too low narrows the effective field of view.
- An **ultrasonic distance sensor** is a common addition for obstacle-avoidance behavior.

The trap: buying the cheapest all-plastic gearbox kit to save a few dollars. Plastic gear teeth strip under real, repeated load in a way a metal gearbox simply doesn't — a cost that shows up weeks later, not at checkout.

Worked example: the same IR sensor mounted at the recommended 5-10mm height gives a clean, reliable high/low distinction between line and background; mounted too high, the same sensor's reading gets noisy and inconsistent.

Lesson 4 — Closing the Loop: PD Control, and Diagnosing Driver Failures

This is the actual point of a rover versus every sensor-only project in this chain: reading a value and acting on it, continuously, in real time.

- The loop's structure: read the sensor error, compute a correction, apply it to the motor speeds, repeat — the fundamental shape of any closed-loop control system.
- A **proportional (P)** correction alone tends to overshoot and oscillate, sometimes with growing swings rather than settling down.
- Adding a **derivative (D)** term — reacting to how fast the error is changing — damps that oscillation out, letting the rover settle onto the line smoothly.
- Too-high Kp even with damping still overshoots; too-low Kp is sluggish and drifts further off the line before correcting at all.

The trap: setting Kp too high with no damping (Kd), causing the rover to oscillate wildly or lose the line entirely. The fix is rarely "just lower Kp" alone — adding real damping (Kd) is usually what actually stabilizes it.

Worked example — three real regimes: Kp=0.15 with no damping grows into wider and wider oscillation over time; the same Kp=0.15 with Kd=0.6 converges cleanly onto the line; pushing Kp up to 1.2 (even with that same damping) diverges outright.

Reflection

Why is "acting on the world" (motors) a fundamentally different problem from "reading it" (sensors), in terms of what can actually go wrong? If you found your rover oscillating right at the edge of the line instead of smoothly tracking it, what would you change about K_p and K_d , and why?